Same Code, Different Contexts: The Eight Study Variants

COMPUTE 2026 Working Group on Refute Problem Difficulty

October 26, 2026

Abstract

This document lists the eight Refute Problem variants used in the study: two problems, each written in a direct and a contextualized form and paired with a context-independent and a context-sensitive bug. For every variant it gives the specification the student reads, the program they are asked to refute, why the form of the specification may matter for that bug, a checked counter-example, and the rule a Step 3 input must satisfy. It closes by explaining how a problem is stored in the study application's database. All programs, inputs and outputs are taken from, and computed by, the application itself.

Contents

1 Design in brief **2**

2 Problem P1: Threshold (heart-rate monitor) **4**

2.1 Shared by all four variants 4

2.2 Variant `P1_Direct_buggy1` 4

2.3 Variant `P1_Contextualized_buggy1` 5

2.4 Variant `P1_Direct_buggy2` 6

2.5 Variant `P1_Contextualized_buggy2` 7

3 Problem P2: Warmest Night (Leh) **9**

3.1 Shared by all four variants 9

3.2 Variant `P2_Direct_buggy1` 9

3.3 Variant `P2_Contextualized_buggy1` 10

3.4 Variant `P2_Direct_buggy2` 11

3.5 Variant `P2_Contextualized_buggy2` 12

4 Database fields of a problem **14**

4.1 Related tables 15

A Code-mixed specifications **17**

A.1 P1, Direct form 17

A.2 P1, Contextualized form 17

A.3 P2, Direct form 18

A.4 P2, Contextualized form 19

1 Design in brief

Each problem has one reference solution and is written in two forms. The *direct* form states the computation in computational terms; the *contextualized* form places the same computation in a real-world setting. Both forms are fully specified and are solved by the same reference solution. Each problem is paired with two buggy programs: a *context-independent* bug (Buggy1), whose failing inputs are no easier to think of in the setting, and a *context-sensitive* bug (Buggy2), which the setting may help students find, either by suggesting realistic inputs that expose it or by giving the faulty code a meaning that looks wrong. Crossing form with bug gives the eight variants of Table 1.

Each student attempts both problems, one in each form, and sees one of each problem's two buggy programs. Which form, which program, and which problem comes first are counterbalanced (Table 2): any four consecutive rows show every form–bug combination of each problem once, and all eight also balance the order.

Within a problem the student works through three steps, with no feedback between them:

Step 1: the task.
The student reads the specification (switching to a code-mixed version if they wish) and gives three input–output pairs. No code is shown. The pairs must all be correct to continue.

Step 2: the code.
The student sees the buggy program, without the specification, and traces it on inputs we supply. None of these inputs is a counter-example.

Step 3: refutation.
The student sees the specification and the program together, with their own Step 1 pairs and Step 2 traces, and gives an input on which the program fails, the output the task requires for it, and the output the program returns. They have at most three attempts.

Table 1: The eight variants, with one counter-example for each bug.

Variant	Form	Bug	Counter-example
`P1_Direct_buggy1`	Direct	Buggy1 (context-independent)	`has_large_change([70, 72, 110], 15)`
`P1_Contextualized_buggy1`	Contextualized	Buggy1 (context-independent)	`has_large_change([70, 72, 110], 15)`
`P1_Direct_buggy2`	Direct	Buggy2 (context-sensitive)	`has_large_change([72, 74], 15)`
`P1_Contextualized_buggy2`	Contextualized	Buggy2 (context-sensitive)	`has_large_change([72, 74], 15)`
`P2_Direct_buggy1`	Direct	Buggy1 (context-independent)	`warmest([-9.8, -12.0, -9.8])`
`P2_Contextualized_buggy1`	Contextualized	Buggy1 (context-independent)	`warmest([-9.8, -12.0, -9.8])`
`P2_Direct_buggy2`	Direct	Buggy2 (context-sensitive)	`warmest([-14.2, -9.8])`
`P2_Contextualized_buggy2`	Contextualized	Buggy2 (context-sensitive)	`warmest([-14.2, -9.8])`

Table 2: Condition rows. Participants at each institute are dealt these rows in shuffled blocks of eight.

Row	P1 form	P1 bug	P2 form	P2 bug	Order
0	Direct	Buggy1	Contextualized	Buggy1	P1 then P2
1	Direct	Buggy2	Contextualized	Buggy2	P1 then P2
2	Contextualized	Buggy1	Direct	Buggy2	P1 then P2
3	Contextualized	Buggy2	Direct	Buggy1	P1 then P2
4	Direct	Buggy1	Contextualized	Buggy1	P2 then P1
5	Direct	Buggy2	Contextualized	Buggy2	P2 then P1
6	Contextualized	Buggy1	Direct	Buggy2	P2 then P1
7	Contextualized	Buggy2	Direct	Buggy1	P2 then P1

2 Problem P1: Threshold (heart-rate monitor)

2.1 Shared by all four variants

The function signature, the reference solution, and the inputs used in Steps 1 and 2 are identical in all four variants of this problem. The Step 2 inputs are the same for both buggy programs, and neither program fails on any of them.

Signature. `def has_large_change(readings, T):`

Reference solution (never shown to students)

```
def has_large_change(readings, T):
    for i in range(len(readings) - 1):
        if abs(readings[i+1] - readings[i]) > T:
            return True
    return False
```

Step 2 tracing inputs. The control-flow question (how many times each line runs) uses the first input; the data-flow question (variable values) uses the second. Reference outputs:

Input	Output
`has_large_change([3, 5, 4], 5)`	`False`
`has_large_change([2, 10, 11], 5)`	`True`

Step 1 worked examples. Revealed, with their correct outputs, only to a student who asks for help after a wrong Step 1 attempt; the student may not then reuse them as their own pairs:

Input	Output
`has_large_change([70, 95, 96], 15)`	`True`
`has_large_change([], 10)`	`False`

2.2 Variant `P1_Direct_buggy1`

Form: Direct. **Bug:** Buggy1, context-independent.

Specification shown to the student (49 words).

> Write a function `has_large_change(readings, T)` that takes a list of integer `readings` and a positive integer `T`. It returns `True` if the absolute difference between some pair of consecutive elements of `readings` is greater than `T`, and `False` otherwise. The list may be empty or contain a single element.

Program shown in Steps 2 and 3

```
def has_large_change(readings, T):
    for i in range(len(readings) - 1):
        if abs(readings[i+1] - readings[i]) > T:
            return True
        else:                                   # <- bug
            return False                        # <- bug
    return False
```

The bug. The `else` branch returns `False` as soon as the first pair is within `T`, so the loop never looks past the first pair. A counter-example needs an unremarkable first pair followed by a large change.

Why the form matters. The requirement the bug violates (*some* pair, *ever* changes) is stated in this form exactly as in the contextualized one.

Counter-example. `has_large_change([70, 72, 110], 15)`: the task requires `True`, the program returns `False`.

Valid Step 3 inputs. Inputs must satisfy the specification's own preconditions.

Validity check (server-side)

```
def valid(readings, T):
    if not isinstance(readings, list) or not all(type(r) is int for r in readings):
        return 'readings must be a list of whole numbers.'
    if type(T) is not int or T <= 0:
        return 'T must be a positive whole number.'
    return None
```

Expected effect. Baseline for this bug.

2.3 Variant `P1_Contextualized_buggy1`

Form: Contextualized. **Bug:** Buggy1, context-independent.

Specification shown to the student (127 words).

> A hospital's patient monitor records a patient's heart rate, in beats per minute (bpm), once every second. A healthy heart rate changes only slightly from one second to the next. The hospital's clinical guidelines say that a nurse must review a recording if the heart rate ever changes by more than `T` bpm between two consecutive readings: a sudden drop can mean the heart is failing, and a sudden rise can mean the patient is in distress. A recording may be empty or contain a single reading, for example if the monitor was just attached. Write a function `has_large_change(readings, T)` that takes the list of readings (whole numbers) and the threshold `T` (a positive whole number), and returns `True` if the recording needs review, and `False` otherwise.

Program shown in Steps 2 and 3

```
def has_large_change(readings, T):
    for i in range(len(readings) - 1):
        if abs(readings[i+1] - readings[i]) > T:
            return True
        else:                                   # <- bug
            return False                        # <- bug
    return False
```

The bug. The `else` branch returns `False` as soon as the first pair is within `T`, so the loop never looks past the first pair. A counter-example needs an unremarkable first pair followed by a large change.

Why the form matters. Nothing about heart rates makes a failing input more or less likely to occur to a student, and the violated requirement is stated as in the direct form. The setting only adds reading load.

Counter-example. `has_large_change([70, 72, 110], 15)`: the task requires `True`, the program returns `False`.

Valid Step 3 inputs. Inputs must also be *plausible in the setting*; students are told so, and an implausible Step 3 input is rejected with the message below and uses up an attempt.

Validity check (server-side)

```
def valid(readings, T):
    if not isinstance(readings, list) or not all(type(r) is int for r in readings):
        return 'readings must be a list of whole numbers.'
    if type(T) is not int or T <= 0:
        return 'T must be a positive whole number.'
    if any(r < 30 or r > 220 for r in readings) or T > 60:
        return ('Your input must be a realistic recording: every heart rate '
                'between 30 and 220 bpm, and T at most 60 bpm.')
    return None
```

Expected effect. No gain over Direct; possibly lower for less fluent readers. Longer time (more to read).

2.4 Variant `P1_Direct_buggy2`

Form: Direct. **Bug:** Buggy2, context-sensitive.

Specification shown to the student (49 words).

> Write a function `has_large_change(readings, T)` that takes a list of integer `readings` and a positive integer `T`. It returns `True` if the absolute difference between some pair of consecutive elements of `readings` is greater than `T`, and `False` otherwise. The list may be empty or contain a single element.

Program shown in Steps 2 and 3

```
def has_large_change(readings, T):
    prev = 0                          # <- bug
    for r in readings:
        if abs(r - prev) > T:
            return True
        prev = r
    return False
```

The bug. `prev = 0` compares the first reading with a value that was never recorded. The program fails only when the first reading itself exceeds `T` while no real consecutive pair does.

Why the form matters. Here 0 looks like a natural starting value, and the program is correct on the small inputs a student is likely to try: on `[1, 2, 3]` with $T = 5$ it correctly returns `False`.

Counter-example. `has_large_change([72, 74], 15)`: the task requires `False`, the program returns `True`.

Valid Step 3 inputs. Inputs must satisfy the specification's own preconditions.

Validity check (server-side)

```
def valid(readings, T):
    if not isinstance(readings, list) or not all(type(r) is int for r in readings):
        return 'readings must be a list of whole numbers.'
    if type(T) is not int or T <= 0:
        return 'T must be a positive whole number.'
    return None
```

Expected effect. Baseline for this bug.

2.5 Variant `P1_Contextualized_buggy2`

Form: Contextualized. **Bug:** Buggy2, context-sensitive.

Specification shown to the student (127 words).

> A hospital's patient monitor records a patient's heart rate, in beats per minute (bpm), once every second. A healthy heart rate changes only slightly from one second to the next. The hospital's clinical guidelines say that a nurse must review a recording if the heart rate ever changes by more than `T` bpm between two consecutive readings: a sudden drop can mean the heart is failing, and a sudden rise can mean the patient is in distress. A recording may be empty or contain a single reading, for example if the monitor was just attached. Write a function `has_large_change(readings, T)` that takes the list of readings (whole numbers) and the threshold `T` (a positive whole number), and returns `True` if the recording needs review, and `False` otherwise.

Program shown in Steps 2 and 3

```
def has_large_change(readings, T):
    prev = 0                          # <- bug
    for r in readings:
        if abs(r - prev) > T:
            return True
        prev = r
    return False
```

The bug. `prev = 0` compares the first reading with a value that was never recorded. The program fails only when the first reading itself exceeds `T` while no real consecutive pair does.

Why the form matters. `prev = 0` now reads as a heart rate of 0 bpm before the first reading, a value that makes no sense for a monitored patient. And any realistic recording exposes the bug, because a plausible heart rate is far larger than a plausible threshold.

Counter-example. `has_large_change([72, 74], 15)`: the task requires `False`, the program returns `True`.

Valid Step 3 inputs. Inputs must also be *plausible in the setting*; students are told so, and an implausible Step 3 input is rejected with the message below and uses up an attempt.

Validity check (server-side)

```
def valid(readings, T):
    if not isinstance(readings, list) or not all(type(r) is int for r in readings):
        return 'readings must be a list of whole numbers.'
    if type(T) is not int or T <= 0:
        return 'T must be a positive whole number.'
    if any(r < 30 or r > 220 for r in readings) or T > 60:
        return ('Your input must be a realistic recording: every heart rate '
                'between 30 and 220 bpm, and T at most 60 bpm.')
    return None
```

Expected effect. Higher correctness than Direct; longer reading but a shorter search for an input.

3 Problem P2: Warmest Night (Leh)

3.1 Shared by all four variants

The function signature, the reference solution, and the inputs used in Steps 1 and 2 are identical in all four variants of this problem. The Step 2 inputs are the same for both buggy programs, and neither program fails on any of them.

Signature. `def warmest(temps):`

Reference solution (never shown to students)

```
def warmest(temps):
    best = temps[0]
    best_day = 1
    for i in range(len(temps)):
        if temps[i] > best:
            best = temps[i]
            best_day = i + 1
    return best_day
```

Step 2 tracing inputs. The control-flow question (how many times each line runs) uses the first input; the data-flow question (variable values) uses the second. Reference outputs:

Input	Output
`warmest([2.5, 6.0, 4.0])`	2
`warmest([1.0, 3.5])`	2

Step 1 worked examples. Revealed, with their correct outputs, only to a student who asks for help after a wrong Step 1 attempt; the student may not then reuse them as their own pairs:

Input	Output
`warmest([-2.0, -9.5, -11.0])`	1
`warmest([1.5, -6.0, 3.0])`	3

3.2 Variant `P2_Direct_buggy1`

Form: Direct. **Bug:** Buggy1, context-independent.

Specification shown to the student (34 words).

> Write `warmest(temps)` that takes a non-empty list of numbers and returns the position of its largest value, counting positions from 1. If the largest value occurs more than once, return the smallest such position.

Program shown in Steps 2 and 3

```
def warmest(temps):
    best = temps[0]
    best_day = 1
    for i in range(len(temps)):
        if temps[i] >= best:      # <- bug
            best = temps[i]
```

```
            best_day = i + 1
    return best_day
```

The bug. Because it uses `>=`, it moves to every later value equal to the best so far, so when the largest value occurs more than once it returns the last position instead of the first. A counter-example needs a tie at the maximum.

Why the form matters. The tie rule (smallest position) is stated explicitly, exactly as in the contextualized form.

Counter-example. `warmest([-9.8, -12.0, -9.8])`: the task requires `1`, the program returns `3`.

Valid Step 3 inputs. Inputs must satisfy the specification's own preconditions.

Validity check (server-side)

```
def valid(temps):
    if not isinstance(temps, list) or len(temps) == 0:
        return 'temps must be a non-empty list of numbers.'
    if not all(type(t) in (int, float) for t in temps):
        return 'temps must be a non-empty list of numbers.'
    return None
```

Expected effect. Baseline for this bug.

3.3 Variant `P2_Contextualized_buggy1`

Form: Contextualized. **Bug:** Buggy1, context-independent.

Specification shown to the student (95 words).

> A weather station in Leh, one of the coldest inhabited places in India, records the lowest temperature of each night in January, in degrees Celsius to one decimal place. The list has at least 1 entry, starts with the night of 1 January and has one entry per night, up to the most recent night recorded. The district administration wants to know which night was the warmest. Write `warmest(temps)` that takes the list and returns the date in January of the warmest night. If more than one night was equally warm, return the earliest date.

Program shown in Steps 2 and 3

```
def warmest(temps):
    best = temps[0]
    best_day = 1
    for i in range(len(temps)):
        if temps[i] >= best:      # <- bug
            best = temps[i]
            best_day = i + 1
    return best_day
```

The bug. Because it uses `>=`, it moves to every later value equal to the best so far, so when the largest value occurs more than once it returns the last position instead of the first. A counter-example needs a tie at the maximum.

Why the form matters. The tie rule (earliest date) is stated explicitly, as in the direct form. The setting may still matter: temperatures to one decimal place make exact ties seem less likely to a student imagining realistic data.

Counter-example. `warmest([-9.8, -12.0, -9.8])`: the task requires `1`, the program returns `3`.

Valid Step 3 inputs. Inputs must also be *plausible in the setting*; students are told so, and an implausible Step 3 input is rejected with the message below and uses up an attempt.

Validity check (server-side)

```
def valid(temps):
    if not isinstance(temps, list) or len(temps) == 0:
        return 'temps must be a non-empty list of numbers.'
    if not all(type(t) in (int, float) for t in temps):
        return 'temps must be a non-empty list of numbers.'
    if (len(temps) > 31 or any(t < -40 or t > 10 for t in temps)
            or any(round(t, 1) != t for t in temps)):
        return ('Your input must be a realistic record: at most 31 nights, '
                'each temperature between -40.0 and 10.0 degrees Celsius, '
                'to one decimal place.')
    return None
```

Expected effect. No gain over Direct; possibly lower for less fluent readers. Longer time (more to read).

3.4 Variant `P2_Direct_buggy2`

Form: Direct. **Bug:** Buggy2, context-sensitive.

Specification shown to the student (34 words).

> Write `warmest(temps)` that takes a non-empty list of numbers and returns the position of its largest value, counting positions from 1. If the largest value occurs more than once, return the smallest such position.

Program shown in Steps 2 and 3

```
def warmest(temps):
    best = 0                        # <- bug
    best_day = 1
    for i in range(len(temps)):
        if temps[i] > best:
            best = temps[i]
            best_day = i + 1
    return best_day
```

The bug. `best = 0` means a value replaces it only if it is above zero. When every value is at or below zero, `best` never changes and the program returns 1.

Why the form matters. With only "numbers" to go on, 0 is a natural-looking starting value, and the program is correct on the positive lists a student is likely to try: on `[3.0, 7.5, 5.0]` it correctly returns 2.

Counter-example. `warmest([-14.2, -9.8])`: the task requires `2`, the program returns `1`.

Valid Step 3 inputs. Inputs must satisfy the specification's own preconditions.

Validity check (server-side)

```
def valid(temps):
    if not isinstance(temps, list) or len(temps) == 0:
        return 'temps must be a non-empty list of numbers.'
    if not all(type(t) in (int, float) for t in temps):
        return 'temps must be a non-empty list of numbers.'
    return None
```

Expected effect. Baseline for this bug.

3.5 Variant `P2_Contextualized_buggy2`

Form: Contextualized. **Bug:** Buggy2, context-sensitive.

Specification shown to the student (95 words).

> A weather station in Leh, one of the coldest inhabited places in India, records the lowest temperature of each night in January, in degrees Celsius to one decimal place. The list has at least 1 entry, starts with the night of 1 January and has one entry per night, up to the most recent night recorded. The district administration wants to know which night was the warmest. Write `warmest(temps)` that takes the list and returns the date in January of the warmest night. If more than one night was equally warm, return the earliest date.

Program shown in Steps 2 and 3

```
def warmest(temps):
    best = 0                        # <- bug
    best_day = 1
    for i in range(len(temps)):
        if temps[i] > best:
            best = temps[i]
            best_day = i + 1
    return best_day
```

The bug. `best = 0` means a value replaces it only if it is above zero. When every value is at or below zero, `best` never changes and the program returns 1.

Why the form matters. Every realistic January record from Leh is below zero, so any record whose warmest night is not 1 January exposes the bug. And `best = 0` now says the warmest night so far was 0 °C, warmer than any night in the record, a value that can never be beaten.

Counter-example. `warmest([-14.2, -9.8])`: the task requires `2`, the program returns `1`.

Valid Step 3 inputs. Inputs must also be *plausible in the setting*; students are told so, and an implausible Step 3 input is rejected with the message below and uses up an attempt.

Validity check (server-side)

```
def valid(temps):
    if not isinstance(temps, list) or len(temps) == 0:
        return 'temps must be a non-empty list of numbers.'
    if not all(type(t) in (int, float) for t in temps):
        return 'temps must be a non-empty list of numbers.'
    if (len(temps) > 31 or any(t < -40 or t > 10 for t in temps)
            or any(round(t, 1) != t for t in temps)):
        return ('Your input must be a realistic record: at most 31 nights, '
                'each temperature between -40.0 and 10.0 degrees Celsius, '
                'to one decimal place.')
    return None
```

Expected effect. Higher correctness than Direct; longer reading but a shorter search for an input.

4 Database fields of a problem

Each variant is stored as one row of the `tasks` table. Table 3 explains every field and says whether, and when, a student sees it; Table 4 shows the values stored for one variant. Fields marked *Never* are not sent to the browser at all.

Table 3: Fields of one problem (`tasks` table).

Field	Type	Seen by student	Meaning
Identity			
`id`	integer	Yes (opaque)	Database key, assigned on insert.
`title`	text	Never	Unique name; for study tasks equal to the variant key. Re-seeding matches existing rows by title, so it never duplicates. Study sessions send the browser a neutral "Problem 1" or "Problem 2" instead.
`language`	text	No	Programming language; always `python`.
Study design			
`variant_key`	text	Never	Which of the eight variants this row is, e.g. `P2_Contextualized_buggy2`. Used by the assignment; null for practice tasks.
`problem_key`	text	Never	`P1` or `P2`.
`form`	text	Yes (Step 3)	`direct` or `contextualized`. Sent to the browser only to decide whether to show the "input must be realistic" notice.
`bug_kind`	text	Never	`buggy1` (context-independent) or `buggy2` (context-sensitive). Analysis only.
Specification			
`description`	text	Steps 1, 3	The English specification (the form's text).
`description_mixed`	JSON	Steps 1, 3	Code-mixed versions keyed by native-language code (`te`, `ta`, `hi`, `gu`, `de`). A student receives only their own language; with no entry, no toggle is shown.
`function_signature`	text	Steps 1, 3	First line of the function. The function name is fixed in the answer boxes, so students type only the arguments.
Programs			
`correct_code`	text	Never	Reference solution. Step 1 pairs and Step 3 required outputs are checked against it.
`buggy_code`	text	Steps 2, 3	The faulty program for this variant.
`buggy_line_numbers`	JSON list	Never	Line(s) carrying the bug. Lets the analysis ask whether the student traced the faulty line correctly.
Step 1			
`io_example_inputs`	JSON list	After a wrong attempt	Two calls whose correct outputs are revealed to a student who asks for help; then barred as the student's own pairs.
Step 2			
`trace_sample_inputs`	JSON list	Step 2	Call(s) for the control-flow question (how many times each line runs). Never a counter-example.
`trace_data_flow_inputs`	JSON list	Step 2	Different call(s) for the data-flow question (variable values, with the execution order given).
`trace_prefill_steps`	integer	Step 2	Rows filled in as a worked example (1).

Field	Type	Seen by student	Meaning
`trace_omit_unchanged_vars`	boolean	No	Whether to drop never-changing variables from the value table (off).
`mutation_line_number`, `mutation_new_line_text`, `mutation_prompt`	int, text, text	—	Optional reflection question from the practice tasks. Empty for all study variants.
Step 3			
`input_validator`	text (Python)	Its messages	Defines `valid(...)` with the task's parameters; returns `None` for a valid input or the message shown to the student. The contextualized version adds plausibility ranges.
`step3_max_attempts`	integer	Yes	Attempt limit (3). Set for study tasks only; it also switches Step 3 to the study format (input, required output, program output).

Table 4: The stored values of `P2_Contextualized_buggy2` (long text fields omitted).

Field	Value
`title / variant_key`	`P2_Contextualized_buggy2`
`problem_key`	`P2`
`form`	`contextualized`
`bug_kind`	`buggy2`
`function_signature`	`def warmest(temps):`
`buggy_line_numbers`	`[2]`
`trace_sample_inputs`	`['warmest([2.5, 6.0, 4.0])']`
`trace_data_flow_inputs`	`['warmest([1.0, 3.5])']`
`io_example_inputs`	`['warmest([-2.0, -9.5, -11.0])', 'warmest([1.5, -6.0, 3.0])']`
`description_mixed`	`{te: ..., ta: ..., hi: ..., gu: ..., de: ...}`
`step3_max_attempts`	`3`
`mutation_*`	`null`

4.1 Related tables

A problem row is the fixed material. What students do is recorded in four other tables:

`participants`
: One row per student: institute, student ID, survey answers (native language, medium of schooling, self-rated English reading from 1 to 5), arrival `slot` at the institute, the `condition_row` it was dealt, and `assigned_task_ids` (the two variants, in presentation order). Institute and student ID are unique together, so a returning student resumes rather than being re-assigned.

`study_sessions`
: One row per student × problem: `participant_id`, `position` (1 or 2), `task_id`, and whether Steps 1–3 are completed.

`step_events`
: The timing and answer log: one row each time a step is shown and each time it is submitted, with a server timestamp, attempt number, correctness, and the full submission in `payload`. Step 1 payloads also record, for analysis only, whether each pair would already have refuted the program; Step 3 payloads record the real outputs and each success condition separately.

`spec_view_events`
: One row for each step's first view of the specification (always English) and one per switch between English and code-mixed, with a timestamp.

A Code-mixed specifications

Draft. These versions must be checked by a native speaker of each language against the English text before the pilot; an ambiguity introduced in translation would produce disputed counter-examples. Technical terms, identifiers and code stay in English; the surrounding prose is in the student's language, written in its own script (German: German prose with English technical terms).

A.1 P1, Direct form

Telugu. `has_large_change(readings, T)` అనే function రాయండి. ఇది integers ఉన్న `readings` అనే list ని, ఒక positive integer `T` ని తీసుకుంటుంది. `readings` లో ఏదైనా ఒక consecutive elements pair మధ్య absolute difference `T` కంటే greater అయితే `True` return చేస్తుంది, లేకపోతే `False` return చేస్తుంది. List empty గా ఉండొచ్చు, లేదా అందులో ఒకే ఒక element ఉండొచ్చు.

Tamil. `has_large_change(readings, T)` என்ற function-ஐ எழுதுங்கள். இது integers கொண்ட `readings` என்ற list-ஐயும், ஒரு positive integer `T`-ஐயும் எடுத்துக்கொள்ளும். `readings`-இல் ஏதாவது ஒரு consecutive elements pair-க்கு இடையிலான absolute difference `T`-ஐ விட greater ஆக இருந்தால் `True` return செய்யும், இல்லையென்றால் `False` return செய்யும். List empty ஆக இருக்கலாம், அல்லது அதில் ஒரே ஒரு element மட்டும் இருக்கலாம்.

Hindi. एक function `has_large_change(readings, T)` लिखिए जो integers की एक list `readings` और एक positive integer `T` लेता है। अगर `readings` के किसी भी consecutive elements के pair का absolute difference `T` से greater है, तो यह `True` return करता है, वरना `False`। List empty हो सकती है या उसमें सिर्फ़ एक element हो सकता है।

Gujarati. `has_large_change(readings, T)` નામનું function લખો, જે integers ની એક list `readings` અને એક positive integer `T` લે છે. જો `readings` ના કોઈ પણ consecutive elements ની pair વચ્ચેનો absolute difference `T` કરતાં greater હોય, તો તે `True` return કરે છે, નહીં તો `False`. List empty હોઈ શકે છે અથવા તેમાં ફક્ત એક જ element હોઈ શકે છે.

German. Schreibe eine function `has_large_change(readings, T)`, die eine list von integer `readings` und einen positive integer `T` nimmt. Sie returnt `True`, wenn die absolute difference zwischen irgendeinem pair von consecutive elements von `readings` greater als `T` ist, und sonst `False`. Die list darf empty sein oder nur ein einziges element enthalten.

A.2 P1, Contextualized form

Telugu. ఒక hospital లోని patient monitor, patient యొక్క heart rate ని beats per minute (bpm) లో, ప్రతి second కి ఒకసారి record చేస్తుంది. Healthy heart rate ఒక second నుండి తర్వాతి second కి కొంచెం మాత్రమే మారుతుంది. ఏవైనా రెండు consecutive readings మధ్య heart rate ఎప్పుడైనా `T` bpm కంటే ఎక్కువ మారితే, nurse ఆ recording ని review చేయాలి అని hospital clinical guidelines చెబుతున్నాయి: sudden drop అంటే heart fail అవుతోందని అర్థం కావచ్చు, sudden rise అంటే patient distress లో ఉన్నారని అర్థం కావచ్చు. Recording empty గా ఉండొచ్చు లేదా అందులో ఒకే ఒక reading ఉండొచ్చు, ఉదాహరణకు monitor ని ఇప్పుడే attach చేసినప్పుడు. Readings list (whole numbers) ని, threshold `T` (ఒక positive whole number) ని తీసుకుని, recording కి review అవసరమైతే `True`, లేకపోతే `False` return చేసే `has_large_change(readings, T)` అనే function రాయండి.

Tamil. ஒரு hospital-இன் patient monitor, ஒரு patient-இன் heart rate-ஐ beats per minute (bpm)-இல், ஒவ்வொரு second-க்கும் ஒரு முறை record செய்கிறது. Healthy heart rate ஒரு second-இலிருந்து அடுத்த second-க்கு கொஞ்சம் மட்டுமே மாறும். ஏதாவது இரண்டு consecutive readings-க்கு இடையில் heart rate எப்போதாவது `T` bpm-ஐ விட அதிகமாக மாறினால், nurse அந்த recording-ஐ review செய்ய வேண்டும் என்று hospital-இன் clinical guidelines சொல்கின்றன: திடீர் drop என்றால் heart fail ஆகிறது என்று அர்த்தமாக இருக்கலாம், திடீர் rise என்றால் patient distress-இல் இருக்கிறார் என்று அர்த்தமாக இருக்கலாம். Recording empty ஆக இருக்கலாம் அல்லது அதில் ஒரே ஒரு reading மட்டும் இருக்கலாம், உதாரணமாக monitor இப்போதுதான் attach செய்யப்பட்டிருந்தால். Readings list (whole numbers) மற்றும் threshold `T` (ஒரு positive whole number) ஆகியவற்றை எடுத்துக்கொண்டு, recording-க்கு review தேவைப்பட்டால் `True`, இல்லையென்றால் `False` return செய்யும் `has_large_change(readings, T)` என்ற function-ஐ எழுதுங்கள்.

Hindi. एक hospital का patient monitor हर second एक बार patient का heart rate, beats per minute (bpm) में, record करता है। एक healthy heart rate एक second से अगले second तक बस थोड़ा-सा ही बदलता है। Hospital की clinical guidelines कहती हैं कि अगर किन्हीं भी दो consecutive readings के बीच heart rate कभी भी `T` bpm से ज़्यादा बदलता है, तो nurse को उस recording को review करना होगा: अचानक drop का मतलब हो सकता है कि heart fail हो रहा है, और अचानक rise का मतलब हो सकता है कि patient distress में है। Recording empty हो सकती है या उसमें सिर्फ़ एक reading हो सकती है, जैसे अगर monitor अभी-अभी लगाया गया हो। एक function `has_large_change(readings, T)` लिखिए जो readings की list (whole numbers) और threshold `T` (एक positive whole number) लेता है, और अगर recording को review की ज़रूरत है तो `True` return करता है, वरना `False`।

Gujarati. એક hospital નું patient monitor દર second એક વાર patient નો heart rate, beats per minute (bpm) માં, record કરે છે. Healthy heart rate એક second થી બીજી second સુધી ફક્ત થોડો જ બદલાય છે. Hospital ની clinical guidelines કહે છે કે જો કોઈ પણ બે consecutive readings વચ્ચે heart rate ક્યારેય `T` bpm કરતાં વધારે બદલાય, તો nurse એ તે recording ને review કરવું જ પડે: અચાનક drop નો અર્થ હોઈ શકે કે heart fail થઈ રહ્યું છે, અને અચાનક rise નો અર્થ હોઈ શકે કે patient distress માં છે. Recording empty હોઈ શકે છે અથવા તેમાં ફક્ત એક જ reading હોઈ શકે છે, ઉદાહરણ તરીકે જો monitor હમણાં જ attach કર્યું હોય. `has_large_change(readings, T)` નામનું function લખો, જે readings ની list (whole numbers) અને threshold `T` (એક positive whole number) લે છે, અને જો recording ને review ની જરૂર હોય તો `True` return કરે છે, નહીં તો `False`.

German. Der patient monitor eines Krankenhauses recordet die heart rate eines Patienten in beats per minute (bpm), einmal pro second. Eine gesunde heart rate ändert sich von einer second zur nächsten nur leicht. Die clinical guidelines des Krankenhauses sagen, dass eine nurse eine recording reviewen muss, wenn sich die heart rate zwischen zwei consecutive readings jemals um mehr als `T` bpm ändert: ein plötzlicher drop kann bedeuten, dass das Herz versagt, und ein plötzlicher rise kann bedeuten, dass der Patient in distress ist. Eine recording darf empty sein oder nur ein einziges reading enthalten, zum Beispiel wenn der monitor gerade erst angeschlossen wurde. Schreibe eine function `has_large_change(readings, T)`, die die list der readings (whole numbers) und den threshold `T` (eine positive whole number) nimmt und `True` returnt, wenn die recording ein review braucht, und sonst `False`.

A.3 P2, Direct form

Telugu. Numbers ఉన్న ఒక non-empty list ని తీసుకుని, దానిలోని largest value యొక్క position ని return చేసే `warmest(temps)` రాయండి; positions ని 1 నుండి లెక్కించాలి. Largest value ఒకటి

కంటే ఎక్కువ సార్లు వస్తే, వాటిలో smallest position ని return చేయండి.

Tamil. Numbers கொண்ட ஒரு non-empty list-ஐ எடுத்துக்கொண்டு, அதன் largest value-இன் position-ஐ return செய்யும் `warmest(temps)`-ஐ எழுதுங்கள்; positions-ஐ 1-இலிருந்து எண்ண வேண்டும். Largest value ஒன்றுக்கு மேற்பட்ட முறை வந்தால், அவற்றில் smallest position-ஐ return செய்யுங்கள்.

Hindi. `warmest(temps)` लिखिए जो numbers की एक non-empty list लेता है और उसकी largest value की position return करता है, positions को 1 से गिनते हुए। अगर largest value एक से ज़्यादा बार आती है, तो ऐसी सबसे छोटी (smallest) position return कीजिए।

Gujarati. `warmest(temps)` લખો, જે numbers ની એક non-empty list લે છે અને તેની largest value ની position return કરે છે; positions ને 1 થી ગણવાની છે. જો largest value એક કરતાં વધુ વાર આવે, તો એમાંથી સૌથી નાની (smallest) position return કરો.

German. Schreibe `warmest(temps)`, das eine non-empty list von numbers nimmt und die position ihres largest value returnt, wobei die positions ab 1 gezählt werden. Wenn der largest value mehr als einmal vorkommt, returne die kleinste (smallest) solche position.

A.4 P2, Contextualized form

Telugu. India లో మనుషులు నివసించే అత్యంత చల్లని ప్రదేశాలలో ఒకటైన Leh లోని ఒక weather station, January లో ప్రతి రాత్రి యొక్క lowest temperature ని degrees Celsius లో, ఒక decimal place వరకు record చేస్తుంది. List లో కనీసం 1 entry ఉంటుంది; అది 1 January రాత్రితో మొదలవుతుంది, ప్రతి రాత్రికి ఒక entry ఉంటుంది, చివరిగా record చేసిన రాత్రి వరకు. ఏ రాత్రి అత్యంత warm గా ఉందో district administration తెలుసుకోవాలనుకుంటోంది. List ని తీసుకుని, అత్యంత warm రాత్రి యొక్క January date ని return చేసే `warmest(temps)` రాయండి. ఒకటి కంటే ఎక్కువ రాత్రులు సమానంగా warm గా ఉంటే, earliest date ని return చేయండి.

Tamil. India-வில் மக்கள் வாழும் மிகக் குளிரான இடங்களில் ஒன்றான Leh-இல் உள்ள ஒரு weather station, January மாதத்தில் ஒவ்வொரு இரவின் lowest temperature-ஐ degrees Celsius-இல், ஒரு decimal place வரை record செய்கிறது. List-இல் குறைந்தது 1 entry இருக்கும்; அது 1 January இரவில் தொடங்கி, ஒவ்வொரு இரவுக்கும் ஒரு entry என்று, கடைசியாக record செய்யப்பட்ட இரவு வரை இருக்கும். எந்த இரவு மிகவும் warm ஆக இருந்தது என்று district administration தெரிந்துகொள்ள விரும்புகிறது. List-ஐ எடுத்துக்கொண்டு, மிகவும் warm ஆன இரவின் January date-ஐ return செய்யும் `warmest(temps)`-ஐ எழுதுங்கள். ஒன்றுக்கு மேற்பட்ட இரவுகள் சமமாக warm ஆக இருந்தால், earliest date-ஐ return செய்யுங்கள்.

Hindi. Leh में, जो India की सबसे ठंडी आबादी वाली जगहों में से एक है, एक weather station January में हर रात का lowest temperature, degrees Celsius में, एक decimal place तक record करता है। List में कम से कम 1 entry है, यह 1 January की रात से शुरू होती है और हर रात की एक entry है, सबसे recent record की गई रात तक। District administration जानना चाहता है कि कौन-सी रात सबसे warm थी। `warmest(temps)` लिखिए जो list लेता है और सबसे warm रात की January की date return करता है। अगर एक से ज़्यादा रातें बराबर warm थीं, तो सबसे पहली (earliest) date return कीजिए।

Gujarati. India ની સૌથી ઠંડી વસવાટવાળી જગ્યાઓમાંની એક, Leh માં એક weather station January ની દરેક રાતનું lowest temperature, degrees Celsius માં, એક decimal place સુધી record કરે છે. List માં ઓછામાં ઓછી 1 entry છે; તે 1 January ની રાતથી શરૂ થાય છે અને દરેક રાત માટે એક entry છે, છેલ્લે record થયેલી રાત સુધી. District administration જાણવા માંગે છે કે કઈ રાત સૌથી warm હતી. `warmest(temps)` લખો, જે list લે છે અને સૌથી warm રાતની January ની date return કરે છે. જો એક કરતાં વધુ રાતો સરખી warm હોય, તો સૌથી પહેલી (earliest) date return કરો.

German. Eine weather station in Leh, einem der kältesten bewohnten Orte Indiens, recordet im January die lowest temperature jeder Nacht, in degrees Celsius auf eine decimal place genau. Die list hat mindestens 1 entry, beginnt mit der Nacht des 1. January und hat eine entry pro Nacht, bis zur zuletzt recordeten Nacht. Die district administration möchte wissen, welche Nacht die wärmste war. Schreibe `warmest(temps)`, das die list nimmt und das date im January der wärmsten Nacht returnt. Wenn mehr als eine Nacht gleich warm war, returne das earliest date.